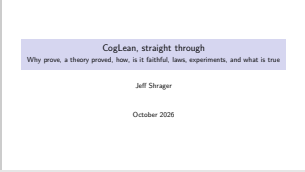
CogLean, straight through

Why prove, a theory proved, how, is it faithful, laws, experiments, and what is true

Jeff Shrager

October 2026

CogLean, straight through

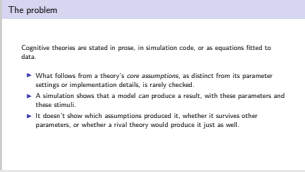


- One straight-through talk, in eight parts, from the key slides of the five CogLean decks. The first slide of each part has a transition in its notes.

The problem

Cognitive theories are stated in prose, in simulation code, or as equations fitted to data.

- What follows from a theory's *core assumptions*, as distinct from its parameter settings or implementation details, is rarely checked.
- A simulation shows that a model *can* produce a result, with these parameters and these stimuli.
- It doesn't show which assumptions produced it, whether it survives other parameters, or whether a rival theory would produce it just as well.



Transition. Part 1, why: what simulation leaves open.

- Most modelling evidence is a fit: one model, one set of parameters, compared with data.
- That's valuable, but it leaves three questions open: which assumptions did the work, is the result robust, and does it discriminate between theories?
- The rest of the talk is about answering those questions with proofs.

Three kinds of evidence a theory can give

One run	**Every run**	**Every model satisfying the postulates**
one model, one parameter setting	one model, all its executions	all models the theory allows
the theory *can* do it	that model *always* does it	the postulates *entail* it
simulation	model checking, sweeps	proof

CogLean is built around the third kind, and keeps the other two in the same language.

CogLean, straight through

Three kinds of evidence a theory can give

Three kinds of evidence a theory can give

One run	Every run	Every model satisfying the postulates
one model, one parameter setting	one model, all its executions	all models the theory allows
the theory *can* do it	that model *always* does it	the postulates *entail* it
simulation	model checking, sweeps	proof

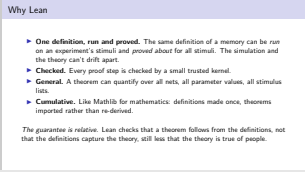
CogLean is built around the third kind, and keeps the other two in the same language.

- A theorem quantified over every model satisfying the postulates says which assumptions matter (the proof uses some and not others), what is robust, and whether a claim is possibility or necessity.
- Simulation is still essential: it explores, suggests conjectures, and fits data. Proof settles which conjectures follow from which assumptions. Experiments with people decide whether the assumptions are right.

Why Lean

- **One definition, run and proved.** The same definition of a memory can be *run* on an experiment's stimuli and *proved about* for all stimuli. The simulation and the theory can't drift apart.
- **Checked.** Every proof step is checked by a small trusted kernel.
- **General.** A theorem can quantify over all nets, all parameter values, all stimulus lists.
- **Cumulative.** Like Mathlib for mathematics: definitions made once, theorems imported rather than re-derived.

The guarantee is relative. Lean checks that a theorem follows from the definitions, not that the definitions capture the theory, still less that the theory is true of people.



- Mathlib is a single, shared, machine-checked library of mathematics. CogLean aims at the same thing for cognitive theory.
- The caveat on this slide matters and will come back: proofs are only as faithful as the formal definitions.

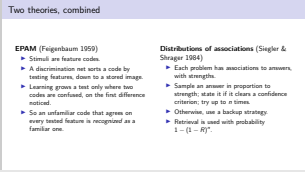
Two theories, combined

EPAM (Feigenbaum 1959)

- Stimuli are feature codes.
- A discrimination net sorts a code by testing features, down to a stored image.
- Learning grows a test only where two codes are confused, on the first difference noticed.
- So an unfamiliar code that agrees on every tested feature is *recognized as* a familiar one.

Distributions of associations (Siegler & Shrager 1984)

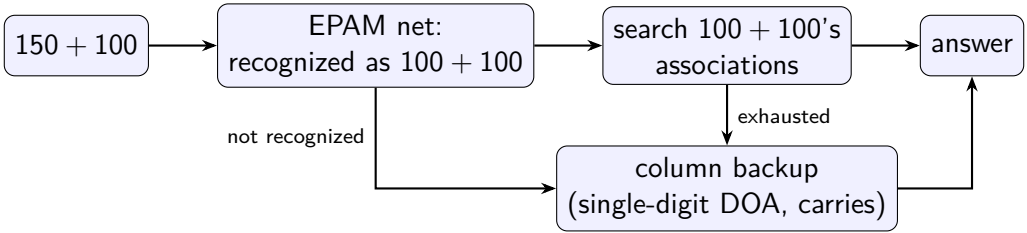
- Each problem has associations to answers, with strengths.
- Sample an answer in proportion to strength; state it if it clears a confidence criterion; try up to n times.
- Otherwise, use a backup strategy.
- Retrieval is used with probability $1 - (1 - R)^n$.



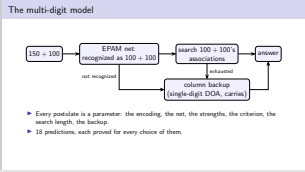
Transition. Part 2, a theory proved: two classic models combined into one of multi-digit addition.

- R is the chance that one search clears the criterion. Both theories' phenomena are already proved in the library.
- In DOA, a flat distribution raises both errors and backup use; they go together. That fact is what the next slides break.

The multi-digit model



- Every postulate is a parameter: the encoding, the net, the strengths, the criterion, the search length, the backup.
- 18 predictions, each proved for every choice of them.



- The column backup is built from single-digit DOA trials chained by carries. Carrying adds no error: a column error is a single-digit error moved to its place.
- The predictions include near misses (within 200 or 20, depending on which digits the net tests), harmless confusions (same sum, commuted problems), and no interference with practiced problems.

Confident errors

Let a familiar problem q have *sharp* associations (only its correct sum clears the criterion), and let x, with a different sum, be recognized as q. Then:

1. on q, whole-problem retrieval never errs;
2. on x, every whole-problem answer is wrong;
3. x uses the backup exactly as rarely as q does: $(1 - R_q)^n$.

In DOA alone, errors and backup use rise together. Confusion breaks the tie: a retrieved, wrong answer with no extra backup use.

Confident errors

Let a familiar problem q have *sharp* associations (only its correct sum clears the criterion), and let x, with a different sum, be recognized as q. Then:

1. on q, whole-problem retrieval never errs;
2. on x, every whole-problem answer is wrong;
3. x uses the backup exactly as rarely as q does: $(1 - R_q)^a$.

In DOA alone, errors and backup use rise together. Confusion breaks the tie: a retrieved, wrong answer with no extra backup use.

- This is the prediction neither theory makes alone.
- Two cautions. The model has no timing, so "fast" would need a timing model. And the errors are frequent only if retrieval on q usually succeeds (R_q near 1); if $R_q = 0$, every trial uses the backup.

A worked case

- Encoding: each digit in four bits (24 features). The net learns $100 + 100$ and $100 + 200$, which differ only in the second addend's hundreds digit, so that is all it tests.
- Associations: .90 on each problem's sum, .01 elsewhere; criterion .50; search length 1.
- $150 + 100$ is recognized as $100 + 100$.

$150 + 100$ answered 200, by retrieval	45/46
$150 + 100$ answered 250, by the columns	1/46

A worked case

- Encoding: each digit in four bits (24 features). The net learns $100 + 100$ and $100 + 200$, which differ only in the second addend's hundreds digit, so that is all it tests.
- Associations: .90 on each problem's sum, .01 elsewhere; criterion .50; search length 1.
- $150 + 100$ is recognized as $100 + 100$.

$150 + 100$ answered 200, by retrieval	45/46
$150 + 100$ answered 250, by the columns	1/46

- These numbers are for illustration, computed exactly by the Lean kernel. The structural result, confident errors, holds for every parameter choice.

In words: Confident errors: the proof is a composition

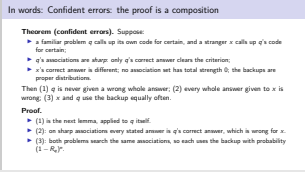
Theorem (confident errors). Suppose:

- a familiar problem q calls up its own code for certain, and a stranger x calls up q's code for certain;
- q's associations are *sharp*: only q's correct answer clears the criterion;
- x's correct answer is different; no association set has total strength 0; the backups are proper distributions.

Then (1) q is never given a wrong whole answer; (2) every whole answer given to x is wrong; (3) x and q use the backup equally often.

Proof.

- (1) is the next lemma, applied to q itself.
- (2): on sharp associations every stated answer is q's correct answer, which is wrong for x.
- (3): both problems search the same associations, so each uses the backup with probability $(1 - R_q)^n$.



Transition. Part 3, how: the proof in words, then the actual Lean.

Confident errors: the proof is a composition

```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    prob (recognizeRetrieve code retrieve str as cc n backup q) (wholeWrong correct q) = 0
    ↪ ∧
      prob (recognizeRetrieve code retrieve str as cc n backup x) (wholeWrong correct x) =
        prob (recognizeRetrieve code retrieve str as cc n backup x) isWhole ∧
      prob (recognizeRetrieve code retrieve str as cc n backup x) isBackup =
        prob (recognizeRetrieve code retrieve str as cc n backup q) isBackup := by
  refine ⟨confusion_gives_neighbours_answer code retrieve str as cc n backup correct q q
  ↪ (hT _) hq hs,
    wrong_eq_whole_of_sharp code retrieve str as cc n backup correct x _ _ (hT _) hx hs
    ↪ hne, ?_⟩
  rw [backup_prob_certain code retrieve str as cc n backup x _ hx hbx hT,
    backup_prob_certain code retrieve str as cc n backup q _ hq hbq hT]
```

CogLean, straight through

Confident errors: the proof is a composition

Confident errors: the proof is a composition



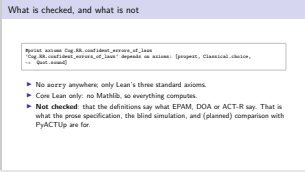
```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (otr y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieves (code q) = pure (some (code q)))
    (hx : retrieves (code x) = pure (some (code q)))
    (hw : #keep otr (≠ (code q) (correct q)) (hne : correct x ≠ correct q) :
    prob (recognizeDet retrieve code retrieve otr as cc x backup q) (hbx)(wrong correct q) = 0 ∧
      prob (recognizeRetrieve code retrieve otr as cc x backup q) (hbx)(wrong correct x) =
        prob (recognizeRetrieve code retrieve otr as cc x backup x) (hbx)hbx ∧
      prob (recognizeRetrieve code retrieve otr as cc x backup x) (hbx)hbq =
        prob (recognizeRetrieve code retrieve otr as cc x backup q) (hbx)hbq := by
  refine ⟨confusion_gives_neighbours_answer code retrieve otr as cc x backup correct q q
    (hT _) hq hx,
    wrong_eq_choice_of_sharp code retrieve otr as cc x backup correct x _ (hT _) hx hx
      hne, _⟩
  rw [backup_prob_certain code retrieve otr as cc x backup x _ hx hbx hT,
    backup_prob_certain code retrieve otr as cc x backup q _ hq hbq hT]
```

- Three conjuncts, three earlier lemmas. The first is the neighbour's-answer lemma applied to q itself; the second says that on sharp associations, every stated answer to x is wrong; the third rewrites both backup probabilities to the same closed form $(1 - R_q)^n$, after which the goal is an identity.
- Note the hypotheses: retrieval is certain (`= pure (some ...)`). That is exactly what ACT-R never satisfies, and what the laws later supply.

What is checked, and what is not

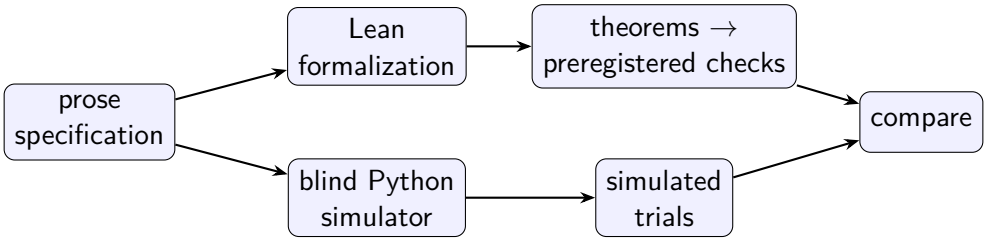
```
#print axioms Cog.RR.confident_errors_of_laws
'Cog.RR.confident_errors_of_laws' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

- No `sorry` anywhere; only Lean's three standard axioms.
- Core Lean only: no Mathlib, so everything computes.
- **Not checked**: that the definitions say what EPAM, DOA or ACT-R say. That is what the prose specification, the blind simulation, and (planned) comparison with PyACTUp are for.



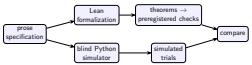
- The three axioms are those every Lean development uses: propositional extensionality, choice, and quotient soundness.
- Code: `CogLib/CogLib/Rep/Distribution.lean`, `Memory/DiscriminationNet.lean`, `Process/SampledRetrieval.lean`, `Process/RecognizeRetrieve.lean`, `Phenomena/RetrievalLaws.lean`; `MultiDigit/MultiDigit/AnyMemory.lean`; `Experiments/Experiments/FaceNaming.lean`.

The idea: two independent readings



N-version programming (Avižienis 1985), applied to a theory. A disagreement means one of three things: the prose is ambiguous, the formalization drifted, or the simulator has a bug.

The idea: two independent readings



N-version programming (Avižienis 1985), applied to a theory. A disagreement means one of three things: the prose is ambiguous, the formalization drifted, or the simulator has a bug.

Transition. Part 4, is it faithful: a blind simulation against preregistered predictions.

- In fault-tolerant software, several teams implement one specification independently, and their disagreements locate the faults. Here the two versions are the Lean formalization and a simulator written without seeing it.
- The Lean side also exports exact outcome distributions for shared instances, so some comparisons are exact, not just statistical.

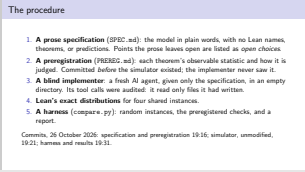
The procedure

1. **A prose specification** (`SPEC.md`): the model in plain words, with no Lean names, theorems, or predictions. Points the prose leaves open are listed as *open choices*.
2. **A preregistration** (`PREREG.md`): each theorem's observable statistic and how it is judged. Committed *before* the simulator existed; the implementer never saw it.
3. **A blind implementer**: a fresh AI agent, given only the specification, in an empty directory. Its tool calls were audited: it read only files it had written.
4. **Lean's exact distributions** for four shared instances.
5. **A harness** (`compare.py`): random instances, the preregistered checks, and a report.

Commits, 26 October 2026: specification and preregistration 19:16; simulator, unmodified, 19:21; harness and results 19:31.

CogLean, straight through

The procedure



- The commit timestamps are the preregistration's evidence: the checks were fixed before any simulator output existed.
- The simulator is committed exactly as delivered. Disagreements are recorded, not fixed in place.

Results: no failures

Check	Lean theorem	kind	tests	failures
Lean exact	`#eval MD.trial`	prob	243	0
P1	`columns_prob`	prob	394	0
P2	`whole_depends_only_on_route`	prob	8543	0
P3	`confusion_gives_neighbours_answer`	exact	200	0
P4	`confident_errors`	exact+prob	222	0
P7	`learning_preserves_trial`	exact	74692	0
P9	`columns_place_value`	exact	248800	0
P11	`discrimination_removes_confusion`	exact	91090	0
P6a, P6b, P14, P17, P18			0	*vacuous*

The prose and the formalization agree on recognition, EPAM learning, retrieval, both backups, and carrying.

CogLean, straight through

Results: no failures

Results: no failures

Check	Lean theorem	kind	tests	failures
Lean exact	#eval RD.trial	prob	243	0
P1	columns_pcnh	prob	394	0
P2	whole_depends_only_on_route	prob	8543	0
P3	confusion_gives_neighbours_answer	exact	200	0
P4	confident_errors	exact+prob	222	0
P7	learning_preserves_trial	exact	74602	0
P9	columns_place_value	exact	248800	0
P10	discrimination_removes_confusion	exact	91090	0
P6a, P6b, P14, P17, P18			0	vacuous

The prose and the formalization agree on recognition, EPAM learning, retrieval, both backups, and carrying.

- A selection of rows; the full table (22 checks) is in `MultiDigit/sim/RESULTS.md`.
- "Lean exact": 243 outcome frequencies on the four shared instances, each within tolerance of Lean's exact probability.

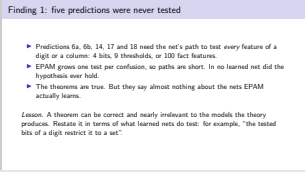
Finding 1: five predictions were never tested

- Predictions 6a, 6b, 14, 17 and 18 need the net's path to test *every* feature of a digit or a column: 4 bits, 9 thresholds, or 100 fact features.
- EPAM grows one test per confusion, so paths are short. In no learned net did the hypothesis ever hold.
- The theorems are true. But they say almost nothing about the nets EPAM actually learns.

Lesson. A theorem can be correct and nearly irrelevant to the models the theory produces. Restate it in terms of what learned nets do test: for example, "the tested bits of a digit restrict it to a set".

CogLean, straight through

Finding 1: five predictions were never tested



- Without the non-vacuity rule, these would have been reported as five passes.
- This is a finding about proving things in cognitive science generally: hypotheses that make a proof go through can quietly exclude the cases of interest.

Finding 3: a prediction depends on an open choice

The predicted learning sequence: a confident error, then column work, then fluent correct retrieval.

O2: new problem starts with	discriminated	canonical	settles on correct	mixed	trapped on error
no associations	48	40	7	1	0
only the correct answer	61	28	33	0	0
a copy of the neighbour's	44	0	2	35	7

- With no associations, the predicted sequence appears (40 of 48).
- Starting with the correct answer, the column phase is often skipped.
- Copying the neighbour's associations perpetuates the error.

Finding 3: a prediction depends on an open choice

The predicted learning sequence: a confident error, then column work, then fluent correct retrieval.

O2 new problem starts with	discriminated	canonical	settles on correct	mixed	trapped on error
no associations	48	40	7	1	0
only the correct answer	61	28	33	0	0
a copy of the neighbour's	44	0	2	35	7

- With no associations, the predicted sequence appears (40 of 48).
- Starting with the correct answer, the column phase is often skipped.
- Copying the neighbour's associations perpetuates the error.

- This part was exploratory, not preregistered: learning over many trials is not yet proved in Lean.
- The model leaves O2 open, and the prediction depends on it. Which reading is right is a substantive question about the theory, now sharply posed.

The generality problem

There are a great many specific models: EPAM, ACT-R, SAM, REM, MINERVA 2, TODAM, connectionist networks...

- Formalizing them one at a time doesn't scale.
- A pile of formalized models doesn't say what any two of them *disagree* about.
- And none of them is "true": unlike mathematics, there's no settled foundation to build on.

The generality problem

There are a great many specific models: EPAM, ACT-R, SAM, REM, MINERVA 2, TODAM, connectionist networks. . .

- Formalizing them one at a time doesn't scale.
- A pile of formalized models doesn't say what any two of them *disagree* about.
- And none of them is "true": unlike mathematics, there's no settled foundation to build on.

Transition. Part 5, generalizing: memory as a plug-in, and a composition reused across tasks.

- This is the central design question. The answer on the next slides is to organize the library around laws, the way Mathlib organizes algebra.

The same problems, two memories

	EPAM	ACT-R
$150 + 100$ answered 200 (the neighbour's sum)	45/46	225/529
$150 + 100$ answered 300 (differs on a tested digit)	0	225/1058
$150 + 100$ answered correctly	1/46	383/1058
errors on the practiced $100 + 100$	0	225/782
the same, after also studying $100 + 300$	0	3375/5842

ACT-R: confusion is graded, nothing is retrieved for certain, and new practice interferes with old.

The same problems, two memories

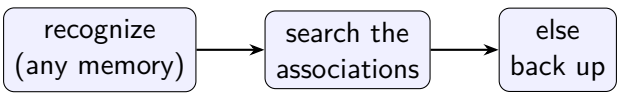
	EPAM	ACT-R
$150 \div 100$ answered 200 (the neighbour's sum)	45/46	225/529
$150 \div 100$ answered 300 (differs on a tested digit)	0	225/1058
$150 \div 100$ answered correctly	1/46	383/1058
errors on the practiced $100 \div 100$	0	225/782
the same, after also studying $100 \div 300$	0	3375/5842

ACT-R: confusion is graded, nothing is retrieved for certain, and new practice interferes with old.

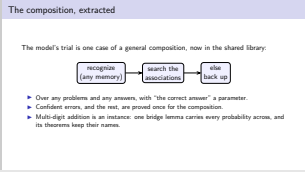
- EPAM's zeros are theorems (no interference; near misses bounded by the tested digits). ACT-R's departures are proved on this instance and explained by laws it lacks.

The composition, extracted

The model's trial is one case of a general composition, now in the shared library:



- Over any problems and any answers, with "the correct answer" a parameter.
- Confident errors, and the rest, are proved once for the composition.
- Multi-digit addition is an instance: one bridge lemma carries every probability across, and its theorems keep their names.



- The multi-digit definitions were left untouched, so the Lean export used by the simulation and the paper is byte-identical.

A second task: naming people

Faces are codes; familiar Ann (000) and Bob (110); stranger Carl (100), one feature from each. The backup is "don't know".

	EPAM	ACT-R
Carl named Bob	45/46	225/391
Carl named Ann	0	225/782
Carl: "don't know"	1/46	107/782
Bob: "don't know"	1/46	229/2254
Bob named Ann	0	225/2254

With EPAM, *confident misnaming* is inherited from the composition's theorem, for every search length. No arithmetic anywhere.

CogLean, straight through

A second task: naming people

A second task: naming people

Faces are codes; familiar Ann (000) and Bob (110); stranger Carl (100), one feature from each. The backup is "don't know".

	EPAM	ACT-R
Carl named Bob	45/46	225/391
Carl named Ann	0	225/782
Carl: "don't know"	1/46	107/782
Bob: "don't know"	1/46	229/2254
Bob named Ann	0	225/2254

With EPAM, *confident misnaming* is inherited from the composition's theorem, for every search length. No arithmetic anywhere.

- With ACT-R, the theorem's hypothesis (Carl's face calls up Bob's for certain) fails, and so does its conclusion: Carl says "don't know" more often than Bob.

Six laws

Certainty	An item just studied is retrieved by itself with probability 1.
Stability	Studying z leaves the chance that y retrieves itself unchanged.
Order-freedom	Swapping the last two study events leaves retrieval unchanged, for every probe.
Recency	Of two new items studied last, a probe equally similar to both retrieves the later more often.
Stored-only	Nothing is retrieved that was never studied.
Determinacy	Every retrieval has a single outcome.

Six laws

Certainty	An item just studied is retrieved by itself with probability 1.
Stability	Studying x leaves the chance that y retrieves itself unchanged.
Order-freedom	Swapping the last two study events leaves retrieval unchanged, for every probe.
Recency	Of two new items studied last, a probe equally similar to both retrieves the later more often.
Stored-only	Nothing is retrieved that was never studied.
Determinacy	Every retrieval has a single outcome.

Transition. Part 6, laws: compare families of models by the laws they have.

- Order-freedom is deliberately narrow: only the last two events are swapped, and only retrieval must agree, not the memory state.
- Determinacy is structural as stated, but each "lacks" cell exhibits two outcomes with positive probability, so it fails any probabilistic version too.

Four families

- **Discrimination** (EPAM): a net of feature tests; learning grows a test where codes are confused.
- **Activation** (ACT-R declarative memory): each use adds activation that decays; partial matching; a failure weight. Decay 1, temperature 1: exactly rational.
- **Strength with a criterion** (after DOA): strengths only add; sample by strength, discounted by mismatch; state if above criterion.
- **Global matching** (MINERVA 2): every study stores a trace; a probe activates all traces by the cube of similarity; memory returns the *echo*, their weighted blend.

Four families

- **Discrimination** (EPAM): a net of feature tests; learning grows a test where codes are confused.
- **Activation** (ACT-R declarative memory): each use adds activation that decays; partial matching; a failure weight. Decay 1, temperature 1: exactly rational.
- **Strength with a criterion** (after DOA): strengths only add; sample by strength, discounted by mismatch; state if above criterion.
- **Global matching** (MINERVA 2): every study stores a trace; a probe activates all traces by the cube of similarity; memory returns the *echo*, their weighted blend.

- Each family is formalized once, with its parameters free. A published model is a choice of parameters within its family.

The matrix

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
Discrimination	**has**	**has**	lacks	lacks	**has**	**has**
Activation	lacks	lacks	lacks	**has**	**has**	lacks
Strength	lacks	lacks	**has**	lacks	**has**	lacks
Global matching	lacks	lacks	**has**	lacks	lacks	**has**

- **has**: a general proof, for every parameter choice meeting stated conditions.
- lacks: a counterexample, computed exactly on the eight 3-bit codes.
- Every law is had by some family and lacked by another; every family has its own row. A script checks both.

The matrix

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
Discrimination	**has**	**has**	lacks	lacks	**has**	**has**
Activation	lacks	lacks	lacks	**has**	**has**	lacks
Strength	lacks	lacks	**has**	lacks	**has**	lacks
Global matching	lacks	lacks	**has**	lacks	lacks	**has**

- **has**: a general proof, for every parameter choice meeting stated conditions.
- lacks: a counterexample, computed exactly on the eight 3-bit codes.
- Every law is had by some family and lacked by another; every family has its own row. A script checks both.

- Examples of “has”: EPAM’s certainty needs only that the items be noticeably different; ACT-R’s recency holds for any positive mismatch discount and failure weight (the later item is retrieved exactly twice as often); strength memory and MINERVA are order-free for every parameter choice.
- A “lacks” cell says only that the law fails somewhere. Mapping the parameter regions where a family does have it is open.

The laws were tested, and failed once

- The first four laws were chosen with three families in view.
- Global matching was added as a test. On those four laws, it had *exactly* the row of strength memory.
- So the laws missed something. What is distinctive about global matching? Its output is a blend.
- Study 110, 101, 011; probe with 111 (their prototype, never studied): MINERVA returns 111.
- Hence the stored-only law, which separates them.

Rule: a law enters only with a witness on each side.

The laws were tested, and failed once

- The first four laws were chosen with three families in view.
- Global matching was added as a test. On those four laws, it had *exactly* the row of strength memory.
- So the laws missed something. What is distinctive about global matching? Its output is a blend.
- Study 110, 101, 011; probe with 111 (their prototype, never studied): MINERVA returns 111.
- Hence the stored-only law, which separates them.

Rule: a law enters only with a witness on each side.

- A law every family satisfies, or satisfies only vacuously, tells theories apart by accident. An earlier all-or-none law, “stable retrieval”, turned out to hold for ACT-R only vacuously, because nothing in ACT-R is retrieved for certain; that is now a theorem.

From laws: which memories give confident errors?

Theorem. Any memory with three laws (*determinacy*, *certainty*, *stability*) produces confident errors whenever a stranger can be taken for a studied item with sharp associations.

- In the matrix of memory families, only discrimination (EPAM) has all three.
- Face naming's confident misnaming is derived this way, from EPAM's matrix cells plus one computed fact (Carl can be taken for Bob).
- The evidence judges that people lack certainty. So explaining confident errors in people needs a graded version: these laws with high probability, not probability 1.

From laws: which memories give confident errors?

From laws: which memories give confident errors?

Theorem. Any memory with three laws (*determinacy*, *certainty*, *stability*) produces confident errors whenever a stranger can be taken for a studied item with sharp associations.

- In the matrix of memory families, only discrimination (EPAM) has all three.
- Face naming's confident misnaming is derived this way, from EPAM's matric cells plus one computed fact (Carl can be taken for Bob).
- The evidence judges that people lack certainty. So explaining confident errors in people needs a graded version: these laws with high probability, not probability 1.

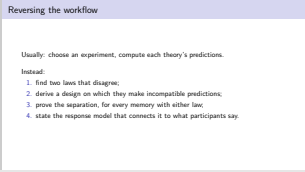
- This is where the composition and the law matrix meet: the matrix now says which families produce the phenomenon.
- The graded theorem is open.

Reversing the workflow

Usually: choose an experiment, compute each theory's predictions.

Instead:

1. find two laws that disagree;
2. derive a design on which they make incompatible predictions;
3. prove the separation, for every memory with either law;
4. state the response model that connects it to what participants say.



Transition. Part 7, experiments from laws: start from a disagreement, derive the design, prove the separation.

- This makes the matrix an experiment generator rather than a scoreboard.

Recency against order-freedom

Design. Two new items a, b; a probe x equally similar to both. Condition AB studies a then b; condition BA studies b then a; both probe with x.

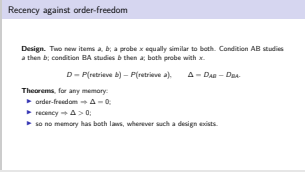
$$D = P(\text{retrieve } b) - P(\text{retrieve } a), \qquad \Delta = D_{AB} - D_{BA}.$$

Theorems, for any memory:

- order-freedom $\Rightarrow \Delta = 0$;
- recency $\Rightarrow \Delta > 0$;
- so no memory has both laws, wherever such a design exists.

CogLean, straight through

Recency against order-freedom



- Recency applied to (a, b) gives $D_{AB} > 0$; applied to (b, a) it gives $D_{BA} < 0$. Order-freedom makes the two conditions' retrieval identical.
- These proofs never mention EPAM, ACT-R, or any parameter.

From retrieval to report

Forced choice: report A if a is retrieved, B if b is; otherwise guess B with probability γ.

$$\Delta_{\text{report}} = \Delta + (2\gamma - 1)\,(o_{AB} - o_{BA}),$$

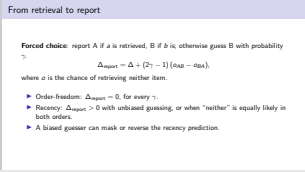
where o is the chance of retrieving neither item.

- Order-freedom: $\Delta_{\text{report}} = 0$, for every γ.
- Recency: $\Delta_{\text{report}} > 0$ with unbiased guessing, or when "neither" is equally likely in both orders.
- A biased guesser can mask or reverse the recency prediction.

CogLean, straight through

From retrieval to report

- The bridge carries its own condition, stated in the theorem: laws plus a response model give an observable contrast.



The same design, four memories

Memory	Δ, no prior item	Δ, after 000	why
ACT-R	$10/17$		recency, so $\Delta > 0$
Strength	0	0	order-free
MINERVA	0	0	order-free
EPAM	0	-2	neither law: computed

- ACT-R's reported effect is $10/17$ for every guessing bias: both orders leave the same chance of "neither".
- EPAM shows the most extreme *primacy* possible. After 000, whichever item is learned first is discriminated from it, and the probe always retrieves that one. Its order effects come from the shape of the net, not from time.

CogLean, straight through

The same design, four memories

- One counterbalanced experiment gives recency, no effect, or primacy, depending on the family, and the matrix says in advance which.

The same design, four memories

Memory	Δ, no prior item	Δ, after 000	why
ACT-R	10/17		recency, so $\Delta > 0$
Strength	0	0	order-free
MINERVA	0	0	order-free
EPAM	0	-2	neither law: computed

- ACT-R's reported effect is 10/17 for every guessing bias: both orders leave the same chance of "neither".
- EPAM shows the most extreme *primacy* possible. After 000, whichever item is learned first is discriminated from it, and the probe always retrieves that one. Its order effects come from the shape of the net, not from time.

What is true here?

In mathematics, a proved theorem is true. Here there are only competing theories. What can a library build on?

Claim	Example	Status
A law entails a prediction	stability $\Rightarrow$ practiced items stay practiced	proved, permanent
A family has a law	discrimination nets have stability	proved, permanent
People have a law	people lack stability in recall	empirical, revisable

Which theory is right follows from the second and third rows together.

What is true here?

In mathematics, a proved theorem is true. Here there are only competing theories. What can a library build on?

Claim	Example	Status
A law entails a prediction	stability ⇒ practiced items stay practiced	proved, permanent
A family has a law	discrimination nets have stability	proved, permanent
People have a law	people lack stability in recall	empirical, revisable

Which theory is right follows from the second and third rows together.

Transition. Part 8, what is true: conditional results, evidence, and limits.

- The conditional results don't expire. They stay true even if every family in the library is refuted.
- Mathlib is in the same position more often than it seems: it proves facts about Euclidean and non-Euclidean geometry alike, and leaves to physics which one space is.

Against the evidence

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
People (judged)	*lacks*	*mixed*	*lacks*	*has*	*mixed*	*mixed*

- A judgment, not a theorem, with sources, kept apart from the proofs.
- Each entry states its **bridge**: how a measure (a report) is taken to show a law (about retrieval).
- Activation fits every settled entry, but each fit rests on unformalized bridges.

Against the evidence

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
People (judged)	*lacks*	*mixed*	*lacks*	*has*	*mixed*	*mixed*

- A judgment, not a theorem, with sources, kept apart from the proofs.
- Each entry states its **bridge**: how a measure (a report) is taken to show a law (about retrieval).
- Activation fits every settled entry, but each fit rests on unformalized bridges.

- Sources: Tulving & Pearlstone 1966 (certainty); McGeoch 1932, Barnes & Underwood 1959, Ratcliff, Clark & Shiffrin 1990 (stability: interference in recall, no list-strength effect in recognition); Murdock 1962 and Glanzer & Cunitz 1966 (order, recency); Posner & Keele 1968 and Roediger & McDermott 1995 (stored-only).
- Stored-only is mixed because endorsing a prototype as old doesn't show the prototype was retrieved; next slide.

Where this is weaker than mathematics

- **Definitions are contestable.** A family is one formalization of a theory. We check that link with blind simulations and community implementations.
- **Laws are idealizations** and presuppose a representation (binary codes, feature-count similarity).
- **Families are flexible.** The informative facts are parameter regions, not single counterexamples.
- **Evidence is never final.** The empirical row will change; the proved rows won't, and the matrix says at once what follows.

Where this is weaker than mathematics

- **Definitions are contestable.** A family is one formalization of a theory. We check that link with blind simulations and community implementations.
- **Laws are idealizations** and presuppose a representation (binary codes, feature-count similarity).
- **Families are flexible.** The informative facts are parameter regions, not single counterexamples.
- **Evidence is never final.** The empirical row will change; the proved rows won't, and the matrix says at once what follows.

- The blind simulation: a fresh agent wrote a Python simulator from a prose specification alone, against predictions preregistered before it existed. It agreed with Lean's exact distributions on every test, and exposed gaps in the formalization.

Next

- **Learning across trials**: feedback updates memory, so memory states become random; multi-digit learning sequences are the target.
- **Graded laws**, and the parameter regions of the "lacks" cells.
- **More families as tests**: REM, a connectionist memory, CHREST, ACT-R with blending.
- **Observational semantics**: decision and report components, so laws about retrieval predict what participants say.
- **Beyond memory**: strategy choice, then production systems.

CogLean, straight through

Next

Next

- **Learning across trials**: feedback updates memory, so memory states become random; multi-digit learning sequences are the target.
- **Graded laws**, and the parameter regions of the "lacks" cells.
- **More families as tests**: REM, a connectionist memory, CHREST, ACT-R with blending.
- **Observational semantics**: decision and report components, so laws about retrieval predict what participants say.
- **Beyond memory**: strategy choice, then production systems.

- The Common Model of Cognition is a useful checklist for which interfaces to add, not a foundation to impose.

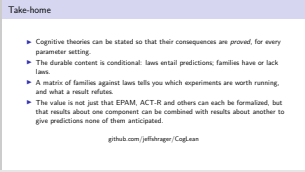
Take-home

- Cognitive theories can be stated so that their consequences are *proved*, for every parameter setting.
- The durable content is conditional: laws entail predictions; families have or lack laws.
- A matrix of families against laws tells you which experiments are worth running, and what a result refutes.
- The value is not just that EPAM, ACT-R and others can each be formalized, but that results about one component can be combined with results about another to give predictions none of them anticipated.

github.com/jeffshrager/CogLean

CogLean, straight through

Take-home



- Two detailed talks follow up: one on recognition, retrieval and confident errors; one on the laws, the matrix and experiments derived from laws.